

AI Banking Assistant

High-Level Architecture & Use Case Description

AI BANKING ASSISTANT — HIGH-LEVEL ARCHITECTURE
& USE CASE DESCRIPTION

Project: AI Banking Assistant
Feature: Multi-Agent Orchestrator
AWS Account: 861976376325
Region: us-east-1
Framework: FastAPI (Python 3.9+)
LLM Provider: Amazon Bedrock (Claude Haiku)
Database: Amazon DynamoDB
Infrastructure: AWS CloudFormation

1. SYSTEM OVERVIEW

The AI Banking Assistant is a conversational AI system built on AWS that helps bank customers with account inquiries and contract lookups. It uses a Multi-Agent Orchestrator pattern where a central intent classifier routes user requests to specialized agents based on the nature of the inquiry.

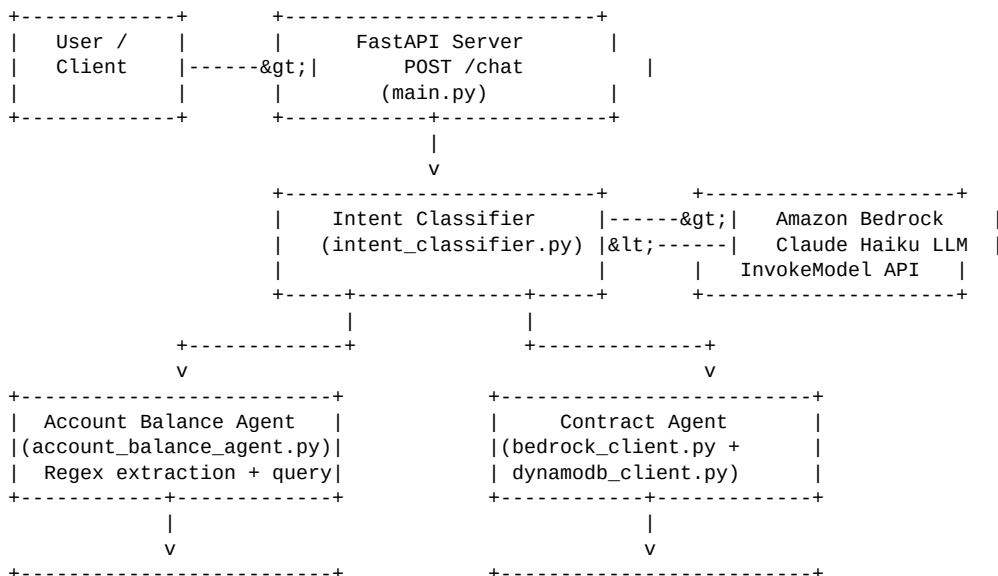
The system leverages:

- Amazon Bedrock (Claude Haiku) for natural language understanding and intent classification
- Amazon DynamoDB for persistent data storage (accounts and contracts)
- AWS CloudFormation for reproducible infrastructure provisioning
- FastAPI for the HTTP API layer

Key Design Principles:

- * Multi-Agent Architecture -- Specialized agents handle distinct domains
- * Intent-Driven Routing -- Bedrock LLM classifies user intent before dispatch
- * Least Privilege -- IAM policies grant only required permissions per agent
- * Idempotent Infrastructure -- CloudFormation ensures reproducible deployments
- * Graceful Degradation -- Failures escalate to human support rather than crashing
- * Structured Logging -- JSON-formatted logs for observability and debugging

2. HIGH-LEVEL ARCHITECTURE



DynamoDB	DynamoDB
Accounts Table	Contracts Table
account_id (PK)	contract_id (PK)
balance, currency,	amount, interest_rate,
account_type	duration
+-----+	+-----+

Data Flow:

1. User sends a message via POST /chat
2. If contract_id is present -> route directly to Contract Agent
3. If no contract_id -> classify intent via Bedrock LLM
4. Route to appropriate agent (balance_inquiry or contract_inquiry)
5. Agent processes request (DB query, optional LLM call)
6. Return structured JSON response (message, summary, status)

3. COMPONENT DESCRIPTIONS

3.1 FastAPI Server (main.py)

The HTTP entry point exposing a POST /chat endpoint. Receives user messages, delegates to the intent classifier for routing, and returns structured JSON responses. Handles backward compatibility for contract_id-based requests. Includes a global exception handler that catches all unhandled errors and returns HTTP 500 with ESCALATE status.

3.2 Intent Classifier (intent_classifier.py)

Uses Amazon Bedrock (Claude Haiku) to classify user messages into one of two intents: balance_inquiry or contract_inquiry. Provides the routing decision for the orchestrator. Includes timeout handling (30s), retry configuration, and structured error propagation via BedrockError.

3.3 Account Balance Agent (account_balance_agent.py)

Specialized agent for handling balance inquiries. Extracts account IDs from user messages using regex pattern matching (ACC-XXXX format), queries the Accounts DynamoDB table via AccountsDynamoDBClient, and returns formatted balance information. If no account ID is found in the message, prompts the user to provide one.

3.4 Contract Agent (bedrock_client.py + dynamodb_client.py)

Handles contract lookup requests. Retrieves contract records from the Contracts DynamoDB table, generates natural language summaries using Bedrock LLM (3 bullet points, max 1024 chars), and returns structured responses. If the contract is not found, returns ESCALATE status.

3.5 DynamoDB -- Accounts Table

Table Name: Accounts
Partition Key: account_id (String)
Billing Mode: PAY_PER_REQUEST (on-demand)
Attributes: account_id, balance (Number), currency (String), account_type (String)
Access Pattern: Single-item lookup by account_id (GetItem)
Client Timeout: 5 seconds

3.6 DynamoDB -- Contracts Table

Table Name: Contracts
Partition Key: contract_id (String)
Attributes: contract_id, amount (Number), interest_rate (Number), duration (String)
Access Pattern: Single-item lookup by contract_id (GetItem)
Client Timeout: 5 seconds

3.7 Amazon Bedrock

Model: anthropic.claude-haiku-4-5-20251001-v1:0
Region: us-east-1
Usage:
(1) Intent classification -- classifies user messages into balance_inquiry or contract_inquiry

(2) Contract summarization -- generates human-readable summaries of contract records
API: InvokeModel with Messages API format (anthropic_version: bedrock-2023-05-31)

3.8 IAM Policy (MultiAgentOrchestratorPolicy)

Managed policy with two statements:

- * AccountsTableReadAccess: dynamodb:GetItem on Accounts table
 - * BedrockIntentClassification: bedrock:InvokeModel on Claude Haiku model
- Follows principle of least privilege.

3.9 CloudFormation Stack (MultiAgentOrchestratorInfra)

Template: infra/cloudformation.yaml
Resources: AccountsTable (DynamoDB) + OrchestratorPolicy (IAM)
Tags: Project=AIBankingAssistant, Feature=MultiAgentOrchestrator
Capabilities: CAPABILITY_NAMED_IAM

4. HIGH-LEVEL USE CASES

USE CASE 1: Account Balance Inquiry

Actor: Bank Customer
Trigger: Customer sends a message asking about their account balance
Precondition: Customer has a valid account (ACC-XXXX format) in the system

Flow:

1. Customer sends message: "What is the balance of ACC-1001?"
2. FastAPI receives POST /chat request (no contract_id)
3. Intent Classifier invokes Bedrock -> classifies as "balance_inquiry"
4. Request routed to Account Balance Agent
5. Agent extracts account ID via regex: ACC-1001
6. Agent queries Accounts DynamoDB table (GetItem)
7. Agent formats and returns balance information

Response:

message: "Account ACC-1001: Balance: 5250.75 USD, Type: savings"
status: "AUTO"

Alternative Flows:

- * No account ID in message -> Agent prompts: "Please provide your account ID"
- * Account not found -> status: "ESCALATE", message: escalating to support
- * DynamoDB unavailable -> HTTP 502, status: "ESCALATE"

USE CASE 2: Contract Lookup & Summary

Actor: Bank Customer
Trigger: Customer provides a contract_id or asks about a contract
Precondition: Contract exists in the Contracts DynamoDB table

Flow:

1. Customer sends message with contract_id in the request body
2. FastAPI detects contract_id presence -- skips intent classification
3. Request routed directly to Contract Agent
4. Agent retrieves contract record from DynamoDB (Contracts table)
5. Agent sends contract details to Bedrock for summarization
6. Bedrock generates 3-bullet-point summary (max 1024 chars)
7. Agent returns contract found message with AI-generated summary

Response:

message: "Contract {id} found and summarized successfully."
contract_summary: "<AI-generated 3-bullet summary>"
status: "AUTO"

Alternative Flows:

- * Contract not found -> status: "ESCALATE", message: escalating to support
- * Bedrock fails -> status: "ESCALATE", message: summary generation failed
- * DynamoDB unavailable -> HTTP 502, status: "ESCALATE"

USE CASE 3: Intent Classification (without contract_id)

Actor: Bank Customer
Trigger: Customer sends a general message without contract_id
Precondition: None (any message is classifiable)

Flow:
1. Customer sends: "Tell me about my contract"
2. No contract_id in request -> intent classification triggered
3. Bedrock classifies as "contract_inquiry"
4. Since no contract_id is available, system prompts user to provide one

Response:
message: "Please provide a contract_id to retrieve your contract details."
status: "AUTO"

Note: This use case demonstrates the routing logic when intent is detected but required context (contract_id) is missing.

USE CASE 4: Graceful Error Handling (Escalation)

Actor: Bank Customer
Trigger: System encounters an error during request processing
Precondition: Any failure condition (service down, timeout, invalid data)

Scenarios:
* Account not found -> ESCALATE with message to contact support
* DynamoDB unavailable -> HTTP 502 with ESCALATE status
* Bedrock timeout (30s) -> ESCALATE with retry suggestion
* Intent classification fails -> ESCALATE with support guidance
* Unexpected error -> Global handler: HTTP 500 with ESCALATE status

Outcome:
Customer always receives a helpful message. System logs full error context (JSON-structured) for debugging. No crashes or unhandled exceptions reach the user. All errors are logged with exception type, message, and traceback.

USE CASE 5: Infrastructure Provisioning (Developer)

Actor: Developer / DevOps Engineer
Trigger: Setting up or verifying the infrastructure environment
Precondition: Valid AWS credentials and access to account 861976376325

Flow (Deploy):
1. Developer runs: python setup_infra.py --action deploy
2. CloudFormation stack deploys DynamoDB Accounts table + IAM policy
3. Bedrock model access is verified (must be ACTIVE)
4. Test data seeded: ACC-1001, ACC-1002, ACC-1003
5. All resources verified independently (4 checks)
6. Colored summary report printed to terminal

Flow (Verify Only):
1. Developer runs: python setup_infra.py --action verify
2. Read-only checks: table status, item count, Bedrock model, IAM policy
3. Summary report printed (no resources modified)

Flow (Seed Only):
1. Developer runs: python setup_infra.py --action seed
2. Test accounts written to DynamoDB (skips existing by default)
3. Result printed with count of items written

Outcome:
Full environment provisioned and validated in a single idempotent command.
Safe to re-run without creating duplicates or errors. Second run returns all COMPLETED or SKIPPED statuses.

5. REQUEST FLOW SUMMARY

```

POST /chat received
|
|-- contract_id present? --YES--&gt; Contract Agent
|
|                                     |-- DynamoDB lookup
|                                     |-- Bedrock summarize
|                                     |-- Return AUTO/ESCALATE
|
|-- contract_id absent? --YES--&gt; Intent Classification
|
|                                     |-- "balance_inquiry"
|                                     |-- Account Balance Agent
|                                     |-- Regex extract
|                                     |-- DynamoDB query
|                                     |-- Return balance
|
|                                     |-- "contract_inquiry"
|                                     |-- Prompt for contract_id

```

6. AWS SERVICES & RESOURCES

Service	Resource	Purpose
DynamoDB	Accounts table	Account balance storage
DynamoDB	Contracts table	Contract record storage
Bedrock	Claude Haiku model	Intent classification + contract summarization
IAM	MultiAgentOrchestratorPolicy	Least-privilege access
CloudFormation	MultiAgentOrchestratorInfra	Infrastructure as code

7. SECURITY CONSIDERATIONS

- * All DynamoDB access uses least-privilege IAM (GetItem only for reads)
- * Bedrock access limited to InvokeModel on a single model ARN
- * Client timeouts (5s DynamoDB, 30s Bedrock) prevent resource exhaustion
- * Input validation: message max 1000 characters (Pydantic enforcement)
- * No secrets in code -- AWS credentials from environment/profile
- * Structured logging excludes sensitive data (no PII in logs)
- * Global exception handler prevents stack traces reaching users

END OF DOCUMENT